

Offline Chatbot for a Python-Based Transmission Line Analysis Software

Complete workflow for two implementation methods:
1) Rule-Based / Hardcoded Chatbot 2) Local LLM Chatbot

Project goal	Add a natural-language interface to an existing transmission-line simulation/calculation program.
Core principle	Keep electrical calculations in Python; use the chatbot to understand questions and present results.
Internet	Method 1: not required. Method 2: can also be fully offline with a locally running LLM and local knowledge base.
Recommended order	Build Method 1 first, then optionally replace its language-understanding layer with Method 2.

Audience: An electrical-engineering project team that needs a practical implementation plan rather than an AI/ML research project.

1. Executive Overview

Your existing program is the engineering engine. It accepts voltage, frequency, line length, conductor type and other operating/design inputs, then calculates quantities such as resistance, inductance, capacitance, losses, voltage regulation and the Ferranti effect. The chatbot sits above this engine.

Recommended principle: language understanding can be handled by rules or an LLM; numerical electrical calculations should remain deterministic Python functions. A language model should not be the authoritative source for exact numerical results when your software can calculate them.

Two methods at a glance

Feature	Rule-Based / Hardcoded	Local LLM
Question understanding	Keywords, phrases, rules	Natural-language model
Training	None	Use existing model; optional RAG/fine-tuning
Numerical calculations	Python functions	Python functions/tools
Subjective explanations	Prewritten rules/templates	LLM + engineering knowledge
Offline	Yes	Yes
Complexity	Low	Medium/High
Control/predictability	Very high	Lower; needs guardrails
Recommended starting point	YES	Later/optional

2. Method 1 — Rule-Based / Hardcoded Chatbot

This method is deterministic and does not use AI. It is the simplest way to add a chatbot to the transmission-line software.

2.1 Architecture

```
User question
↓
Text normalization
↓
Intent detection (keywords / phrase scoring)
↓
Required-input check
↓
Python engineering function
↓
Result + predefined response template
↓
Chat window
```

2.2 Centralize the project variables

```
line_data = {
    "voltage_kv": 135,
    "frequency_hz": 50,
    "distance_km": 150,
    "conductor": "ACSR",
    "current_a": 250
}
```

When the user changes an input in the main software, update this shared state. The chatbot should read the same current state rather than keeping a second copy.

2.3 Convert calculations into functions

```
def calculate_inductance(line_data):
    # your existing engineering formula
    return inductance

def calculate_capacitance(line_data):
    return capacitance

def calculate_losses(line_data):
    return loss_percent

def calculate_pole_count(line_data):
    return pole_count
```

Keep the engineering engine separate from the chatbot. This makes the calculations testable and lets the same functions be used by both chatbot methods.

2.4 Create intents

An intent is the type of question the user is asking. Start with about 10–20.

```
INTENTS = {
    "LINE_LOSS": ["line loss", "transmission loss", "power loss", "loss percentage"],
    "INDUCTANCE": ["inductance", "line inductance"],
    "CAPACITANCE": ["capacitance", "line capacitance"],
    "POLE_COUNT": ["how many poles", "number of poles", "poles required"],
    "POLE_TYPE": ["type of pole", "which pole", "preferred pole"],
    "FERRANTI_EFFECT": ["ferranti effect", "receiving end voltage"],
    "EFFICIENCY": ["efficiency", "line efficiency"],
    "VOLTAGE_REGULATION": ["voltage regulation", "regulation"]
}
```

2.5 Normalize text

```
import re

def clean_text(text):
    text = text.lower()
```

```
text = re.sub(r'^a-z0-9\s|', '', text)
return text.strip()
```

2.6 Detect intent

```
def detect_intent(question):
    q = clean_text(question)

    if "pole" in q and ("how many" in q or "number" in q or "required" in q):
        return "POLE_COUNT"

    if "pole" in q and ("type" in q or "which" in q or "preferred" in q):
        return "POLE_TYPE"

    if "loss" in q:
        return "LINE_LOSS"

    if "inductance" in q:
        return "INDUCTANCE"

    if "capacitance" in q:
        return "CAPACITANCE"

    return "UNKNOWN"
```

For a larger system, use keyword groups and a score for each intent instead of a very long if/elif chain.

2.7 Map intents to engineering functions

```
HANDLERS = {
    "LINE_LOSS": calculate_losses,
    "INDUCTANCE": calculate_inductance,
    "CAPACITANCE": calculate_capacitance,
    "POLE_COUNT": calculate_pole_count
}

intent = detect_intent(user_question)

if intent in HANDLERS:
    result = HANDLERS[intent](line_data)
```

2.8 Add variables to predefined answers

```
def loss_response(loss, line_data):
    limit = 5
    status = "above" if loss > limit else "below"

    return (
        f"For the current {line_data['voltage_kv']} kV, "
        f"{line_data['distance_km']} km line, the calculated "
        f"loss is {loss:.2f}%, which is {status} the {limit}% criterion."
    )
```

The sentence is predefined; the values are inserted dynamically. You can use the same approach for voltage, distance, frequency, conductor, inductance, capacitance, loss, efficiency and other results.

2.9 Subjective questions

For conceptual questions, create a small engineering relationship/rule database. Example: $X_L = 2\pi fL$, so increasing inductance increases inductive reactance. However, resistive loss is $P_{\text{loss}} = I^2R$; inductance does not directly appear in that basic equation. The actual effect on operating loss depends on how the changed reactance affects current, reactive power and voltage.

```
RULES = {
    "INDUCTANCE_INCREASE": {
        "direct_effect": "Inductive reactance increases:  $X_L = 2\pi fL$ .",
        "loss_note": "Inductance does not directly appear in  $P_{\text{loss}} = I^2R$ .",
        "effects": [
            "voltage regulation can change",
            "reactive power behavior can change",
            "current can change depending on operating conditions"
        ]
    }
}
```

2.10 What-if analysis

For a question such as “What happens if I increase the line length from 150 km to 250 km?”, run the same engineering model for both states and compare the outputs.

```
old_data = line_data.copy()
new_data = line_data.copy()
new_data["distance_km"] = 250

old_result = analyze_line(old_data)
new_result = analyze_line(new_data)

comparison = {
    "loss_change": new_result["loss"] - old_result["loss"],
    "inductance_change": new_result["inductance"] - old_result["inductance"]
}
```

The same pattern can be used for a changed inductance, frequency, conductor or another parameter, provided your model defines how that parameter enters the calculation.

2.11 Simple local chat interface

```
while True:
    question = input("You: ")

    if question.lower() == "exit":
        break

    intent = detect_intent(question)
    answer = answer_question(intent, line_data)

    print("Bot:", answer)
```

Once the console version works, connect the same backend to Tkinter, PySide/PyQt, Streamlit or the GUI you already use.

2.12 Complete workflow

[illegible]

3. Method 2 — Local LLM Chatbot

This method uses a locally running language model. It can understand more natural wording, handle follow-up questions and combine numerical results with conceptual explanations. It is more complex than the rule-based method.

3.1 Do not train an LLM from scratch

- Use an existing small open-weight language model.
- Run it locally with a local model runtime such as Ollama or llama.cpp.
- Connect the model to your Python functions.
- Add RAG later if you want the model to use your own local engineering documents.
- Fine-tuning is optional and should not be the first step.

3.2 Suggested stack

- Python — your existing engineering application.
- Ollama or llama.cpp — local model runtime.
- A small local open-weight model appropriate for your computer.
- Optional local RAG/vector-search stack for PDFs, notes and project documentation.
- Tkinter/PySide/Streamlit — optional user interface.

3.3 Architecture

```
User
↓
Local LLM
↓
Understand question + identify needed parameters
↓
Choose an approved engineering function/tool
↓
Python calculation engine
↓
Structured numerical result
↓
Local LLM
↓
Natural-language explanation
↓
User
```

3.4 Expose engineering functions as tools

```
def calculate_losses(distance_km, voltage_kv, conductor, current_a):
    # your deterministic engineering model
    return {
        "loss_percent": loss_percent,
        "loss_kw": loss_kw
    }

def calculate_inductance(distance_km, conductor, frequency_hz):
    return {
        "inductance_h": inductance_h,
        "reactance_ohm": reactance_ohm
    }
```

Give the LLM a description of each approved tool, including its purpose, arguments, units and returned values. The actual calculation remains inside Python.

3.5 Current project state

```
current_state = {
    "voltage_kv": 135,
```

```

"frequency_hz": 50,
"distance_km": 150,
"conductor": "ACSR",
"current_a": 250
}

```

This lets the assistant understand references such as “my line” or “the current line.” Keep this state structured and authoritative.

3.6 Tool-calling example

```

User:
"How much loss does my current line have?"

LLM decision:
tool = calculate_losses

arguments:
distance_km = 150
voltage_kv = 135
conductor = "ACSR"
current_a = 250

Python result:
loss_percent = 4.2

LLM:
"The calculated loss is approximately 4.2% ..."

```

3.7 Subjective/causal questions

For “What happens if I increase inductance?”, the LLM identifies the changed variable, operation and target. It can retrieve an engineering relationship and, if a numerical change is specified, call a what-if calculation.

```

Question:
"What happens to losses if I increase inductance?"

Interpretation:
changed variable = inductance
operation = increase
target = losses

Knowledge:
 $XL = 2\pi fL$ 
 $P_{loss} = I^2R$ 

Action:
1. Explain the direct relationship.
2. Determine whether the operating model changes.
3. If a new numerical value is given, run a modified case.
4. Compare old and new results.
5. Explain the engineering reason.

```

3.8 What-if tool

```

def what_if_change(parameter, new_value, line_data):
    modified = line_data.copy()
    modified[parameter] = new_value
    return analyze_line(modified)

```

Use an allow-list of editable parameters and validate units/ranges before executing changes.

3.9 Local engineering knowledge with RAG

RAG means Retrieval-Augmented Generation. Put approved local material—notes, project documentation, formulas and explanations—into a searchable local knowledge base. Retrieve relevant passages for a question and supply them as context to the local LLM.

```

PDFs / Notes / Documentation
↓
Text extraction
↓
Chunking

```

```
↓
Local embeddings/index
↓
User question
↓
Retrieve relevant passages
↓
Local LLM
↓
Answer
```

RAG is usually more practical than fine-tuning for project documentation because documents can be updated without retraining the model.

3.10 Conversation context

The system can support follow-ups such as: “My line is 150 km.” → “What if I increase it to 250 km?” → “Why did the loss change?” Preserve structured project state and calculation results; do not rely only on free-form chat history.

3.11 Guardrails

- Validate every tool argument and unit.
- Only expose approved engineering functions.
- Never allow model-generated text to execute arbitrary Python code.
- Ask for missing required parameters rather than guessing.
- Use Python's numerical result as the source of truth.
- Make assumptions visible in engineering answers.
- Log the question, selected tool, inputs and result for debugging.

3.12 Complete local-LLM workflow

```
INPUT PANEL
↓
Current structured line state
↓
User natural-language question
↓
Local LLM
↓
Does it need a tool?
■■■ No → retrieve knowledge / explain
■■■ Yes
↓
Select approved Python function
↓
Validate arguments
↓
Run engineering calculation
↓
Return structured result
↓
LLM explains result
↓
Chat window
```

4. Handling Numerical, Subjective and What-If Questions

4.1 Direct numerical question

User: “What is the inductance of my line?”

```
Question
↓
Identify INDUCTANCE
↓
Read current line_data
↓
calculate_inductance(...)
↓
Get numerical result
↓
Insert into answer template
↓
Display
```

4.2 Conceptual question

User: “Why does increasing inductance increase inductive reactance?”

```
Question
↓
Identify concepts: inductance + reactance
↓
Retrieve relationship:  $X_L = 2\pi f L$ 
↓
Explain proportional relationship
↓
Mention relevant assumptions
```

4.3 Causal question

User: “What happens to losses if I increase inductance?”

```
Identify:
changed variable = L
target = losses
operation = increase

Use relationships:
 $X_L = 2\pi f L$ 
 $P_{\text{loss}} = I^2 R$ 

Do not claim a universal direct loss increase.
If a new numerical value or operating condition is available:
run original case
run modified case
compare
explain.
```

4.4 What-if question

User: “Change the line length from 150 km to 250 km and tell me what changes.”

```
1. Identify parameter and new value.
2. Copy the current state.
3. Change only the requested parameter.
4. Run the same engineering model.
5. Compare old and new values.
6. Return a comparison table.
7. Explain the physical reason for the changes.
```

4.5 Missing input

If a result requires a parameter that is not available, ask for it or state that the calculation cannot be completed. Do not invent an engineering value.

5. Recommended Project Folder Structure

```

transmission_project/
■
■ ■ ■ ■ main.py
■ ■ ■ ■ ui.py
■
■ ■ ■ ■ chatbot/
■ ■ ■ ■ rules.py
■ ■ ■ ■ intents.py
■ ■ ■ ■ responses.py
■ ■ ■ ■ llm_agent.py
■
■ ■ ■ ■ engineering/
■ ■ ■ ■ line_parameters.py
■ ■ ■ ■ losses.py
■ ■ ■ ■ inductance.py
■ ■ ■ ■ capacitance.py
■ ■ ■ ■ ferranti.py
■ ■ ■ ■ voltage_regulation.py
■ ■ ■ ■ pole_analysis.py
■
■ ■ ■ ■ knowledge/
■ ■ ■ ■ relationships.json
■ ■ ■ ■ documents/
■
■ ■ ■ ■ tests/
■ ■ ■ ■ test_engineering.py
■
■ ■ ■ ■ config/
■ ■ ■ ■ limits.json

```

Both chatbot methods should call the same engineering functions. Only the language-understanding layer changes.

6. Development Roadmap

```

PHASE 1 – Engineering engine
↓
Central line_data state
↓
Clean, tested calculation functions

PHASE 2 – Rule-based chatbot
↓
10-20 intents
↓
Keyword/synonym matching
↓
Response templates
↓
Missing-input handling

PHASE 3 – What-if analysis
↓
Old state vs new state
↓
Comparison + explanation

PHASE 4 – Local LLM
↓
Install local runtime
↓
Connect model to existing Python functions
↓
Tool calling
↓
Natural-language follow-up

PHASE 5 – Local engineering knowledge
↓
RAG over project notes/PDFs
↓
Conceptual answers grounded in approved material

PHASE 6 – Testing and polish
↓
GUI + logs + error handling + documentation

```

Best starting point: finish the rule-based version first. The local LLM is an advanced extension, not a prerequisite.

7. Testing Strategy

Test	Example	Expected behavior
Numerical	Calculate loss	Exact Python result
Synonym	How many structures?	Recognize POLE_COUNT if configured
Missing input	Calculate loss without required current	Request/flag missing input
What-if	Increase length to 250 km	Run modified case and compare
Conceptual	Why does L affect XL?	Explain $XL = 2\pi fL$
Ambiguous	What happens if I increase it?	Ask what "it" means if context is insufficient
Invalid	Negative line length	Reject or validate input

8. Engineering Design Notes and Guardrails

- Use explicit units: distance_km, voltage_kv, frequency_hz, current_a, resistance_ohm, etc.
- Keep assumptions visible.
- Distinguish direct formula relationships from system-level effects.
- For pole/tower counts, clearly identify the spacing/design assumptions.
- Do not let the chatbot silently change engineering inputs.
- Treat Python calculation output as authoritative for numerical values.
- For recommendations, distinguish a project heuristic from a formal design standard.
- Store test cases so the team can prove that chatbot answers match the engineering engine.

9. Final Recommendation

Start with the rule-based chatbot. Build a clean modular backend around your existing transmission-line calculation engine, centralize the variables, create intents, connect each intent to a calculation or engineering rule, and add dynamic response templates. Then implement what-if analysis.

After that works, add a local LLM as an additional language-understanding layer. The LLM can interpret natural questions, select your approved Python functions, use a local engineering knowledge base for conceptual questions, and explain calculated results.

Core design principle: *LLM for language and explanation; Python for mathematics; local documents for project knowledge.*